

CS 112

Introduction to Data Structures

WEEK 05

Generic Containers

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 The problem with `typedef`
- 2 Class templates
- 3 Turning a class into a template
- 4 STL `vector`
- 5 Iterators
- 6 How long do operations take?
- 7 Amortized constant time **KEY**

One Class, One Type

```
typedef double Item;           // in PyList.h

PyList scores;                 // holds doubles ✓
PyList names;                  // also doubles x
```

Change the `typedef` and every `PyList` in the program changes with it. Want a `PyList` of doubles and a `PyList` of strings in one program? You'd need two copies of the class with different names.

Python never had this problem, a list holds anything. C++ needs types at compile time, so it needs another way.

A Class That Takes a Type

```
template <typename Item>
class PyList {
public:
    void append(const Item &it);
private:
    Item      *myArray;
    unsigned  mySize;
};
```

```
PyList<double>  scores;    // a PyList of doubles
PyList<string>  names;     // a PyList of strings
```

You instantiate a *class* to get an object. You instantiate a *class template* to get a class. `PyList<double>` and `PyList<string>` are two different, separately compiled classes.

Turning a Class Into a Template

- Build and debug it first with `typedef double Item;`: get it working
- Replace the typedef with `template <typename Item>` above the class
- In the `.cpp`, put that same line above **every** method
- Change each `PyList::` to `PyList<Item>::`
- Move the implementation into the header, then `#include` it

```
template <typename Item>
void PyList<Item>::append(const Item &it) { ... }
```

Debug once as a concrete class, then generalise. Templated compiler errors are famously unfriendly, don't meet them and your own bugs at the same time.

TALK TO YOUR NEIGHBOR · TTYN

Why does a class template's implementation go in the header?

- A.** Style, the committee preferred it that way
- B.** The compiler must see the code to generate a class for each type used
- C.** Headers compile faster than source files
- D.** So the linker can find it

You Already Have One: **vector**

```
#include <vector>

vector<int> scores;

scores.push_back(90);           // append
scores.push_back(85);

cout << scores.size();          // 2
cout << scores[0];              // 90
```

The STL is a library of class templates: **vector**, **list**, **stack**, **queue**, **set**, **map**. You've just built a simplified **vector**: from here on you may use the real one.

What **vector** Gives You

Size and room

- `size()` how many are in it
- `capacity()` how many fit before it grows
- `empty()` is it size 0
- `reserve(n)` ask for room up front
- `resize(n)` change the size itself

Reaching an element

- `v[i]` no bounds check
- `at(i)` checked, throws if out of range
- `front()`, `back()` the ends

Adding and removing

- `push_back(x)` our `append`
- `pop_back()` drop the last
- `insert(pos, x)` put one anywhere
- `erase(pos)` take one out
- `clear()` empty it

Positions

- `begin()`, `end()` walk it forwards
- `rbegin()`, `rend()` walk it backwards

Every one of these is something `PyList` either has or could have. The list is worth keeping:

w3schools.com/cpp/cpp_ref_vector.asp

Iterators

```
vector<string>::iterator it;

for (it = names.begin(); it != names.end(); ++it) {
    cout << *it << endl;    // dereference, like a pointer
}
```

- `begin()` : points at the first element
- `end()` : points *just past* the last one
- Every STL container is walked the same way

An iterator behaves like a pointer on purpose: `*it` to read, `++it` to advance.

Walking a Vector

1

`begin()`

is the first element.

`end()`

is the position after the last.

```
vector<string>::iterator it;
for (it = stds.begin(); it != stds.end(); ++it)
    cout << *it << " ";
```

`vector<string> stds`



Walking a Vector

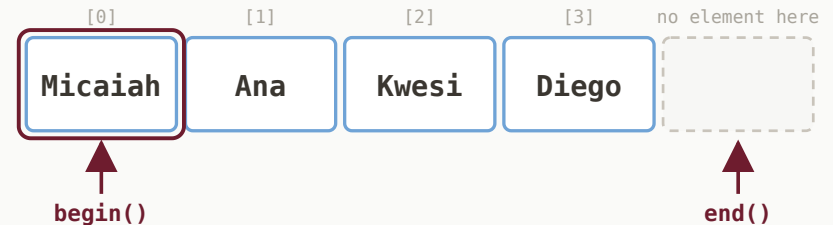
2

```
it =  
stds.begin()
```

stands the iterator on the first element.

```
vector<string>::iterator it;  
for (it = stds.begin(); it != stds.end(); ++it)  
    cout << *it << " ";
```

vector<string> stds



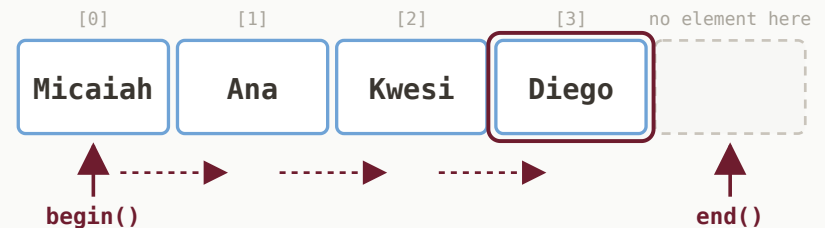
it is a position, not a value

Walking a Vector

3 `*it` is the value there, and `++it` moves it along.

```
vector<string>::iterator it;
for (it = stds.begin(); it != stds.end(); ++it)
    cout << *it << " ";
```

`vector<string> stds`



++it moves to the next position
Micaiah Ana Kwesi Diego

Walking a Vector

4 `it` reaches `end()`, the test is false, and the loop stops.

```
vector<string>::iterator it;  
for (it = stds.begin(); it != stds.end(); ++it)  
    cout << *it << " ";
```

`vector<string> stds`



*it == stds.end(), so the test is false
nothing here was ever read*

Walking a Vector

- 5 An empty vector needs no special case:

`begin()` is already `end()` .

```
vector<string>::iterator it;
for (it = stds.begin(); it != stds.end(); ++it)
    cout << *it << " ";
```

`vector<string> empty`

no element here



`begin() == end()`

the loop body never runs, and no test was needed to arrange it

Counting the Work: Indexing

```
v[i] → *(myArray + i * sizeof(Item))
```

- one multiply, one add, one dereference
- three operations: **whatever i is**
- whatever the size of the array is

Work that doesn't grow with n is **constant time**.
Indexing an array is the classic example.

Counting the Work: Append

Room left

```
myArray[mySize] = it;    // 1 store  
++mySize;                // 1 add
```

2 operations, whatever n is

Full

```
new Item[2 * n]          // room for 2n items  
copy n items             // 3 ops each  
delete [] old            // give it back  
myArray = bigger         // repoint
```

$3n + 5$: three per item copied, plus five that do not depend on n (allocate, store, delete, repoint, count)

So which is it?

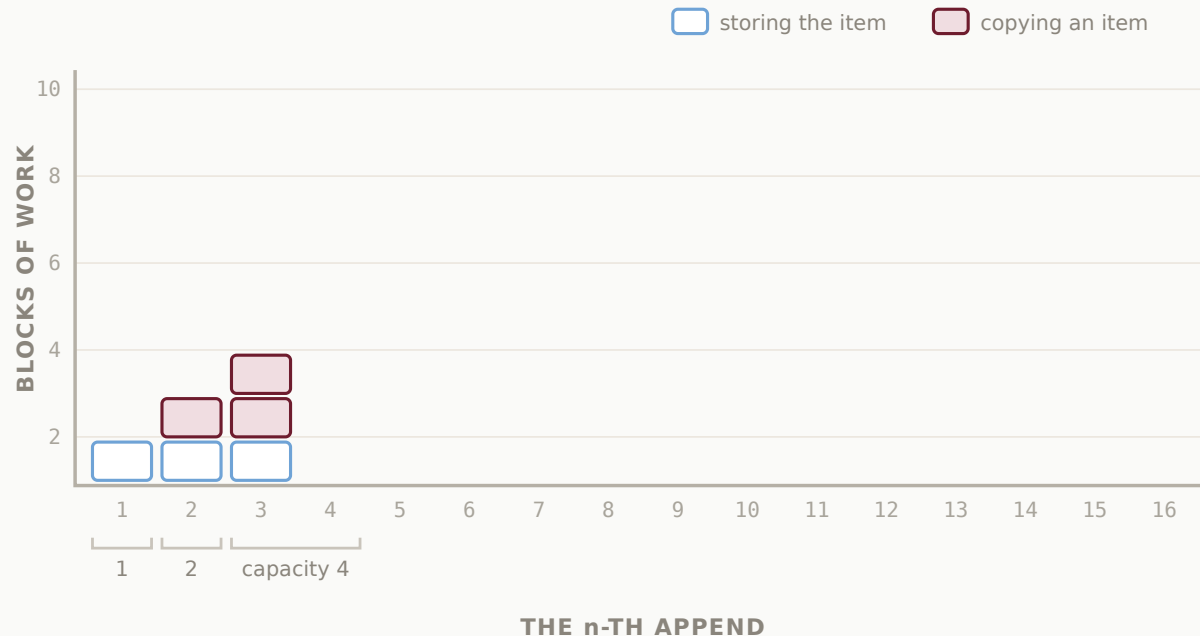
Amortized Constant Time

1 Append 1: there is room, so it just stores the item.



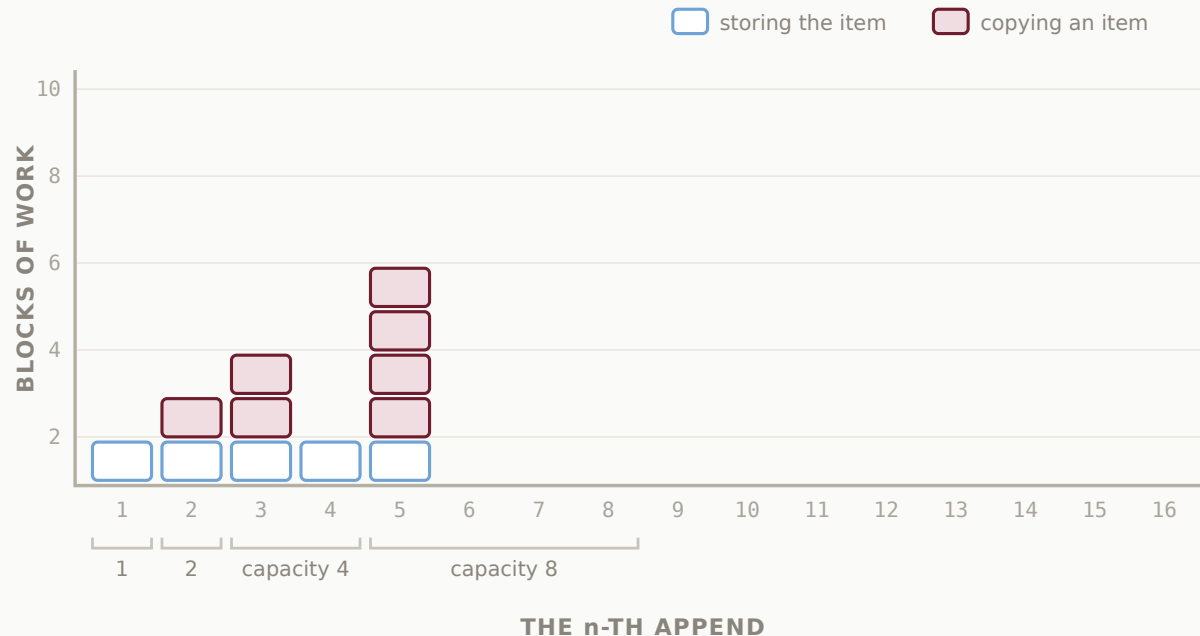
Amortized Constant Time

2 Append 3: the array is full, so it copies 2 items first.



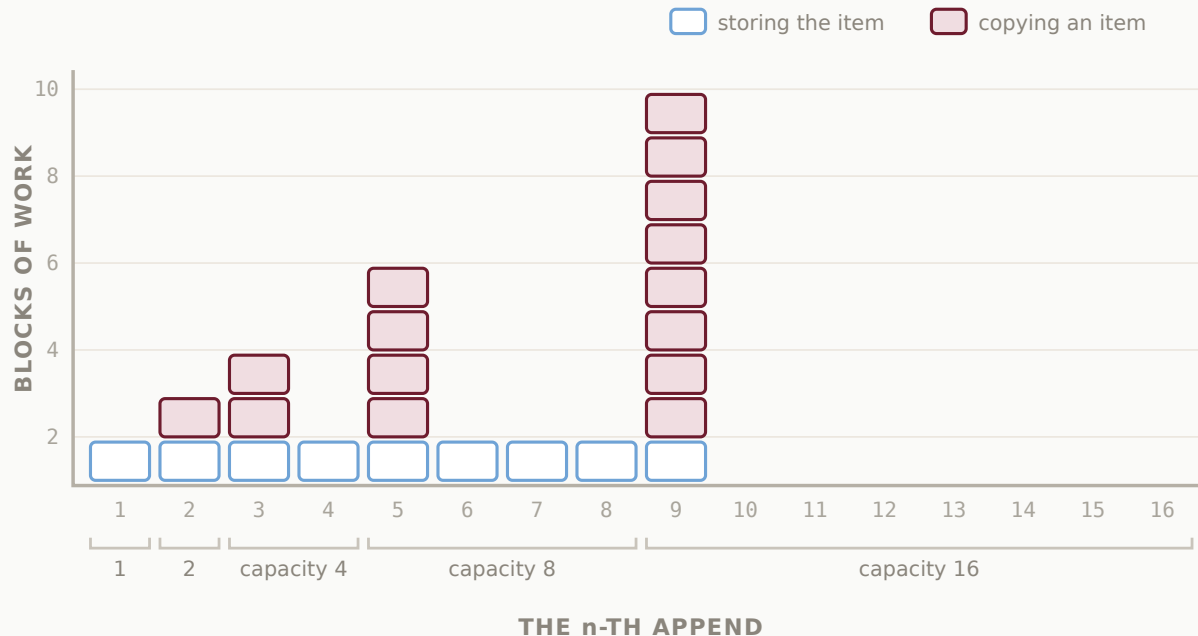
Amortized Constant Time

3 Append 5: the array is full, so it copies 4 items first.



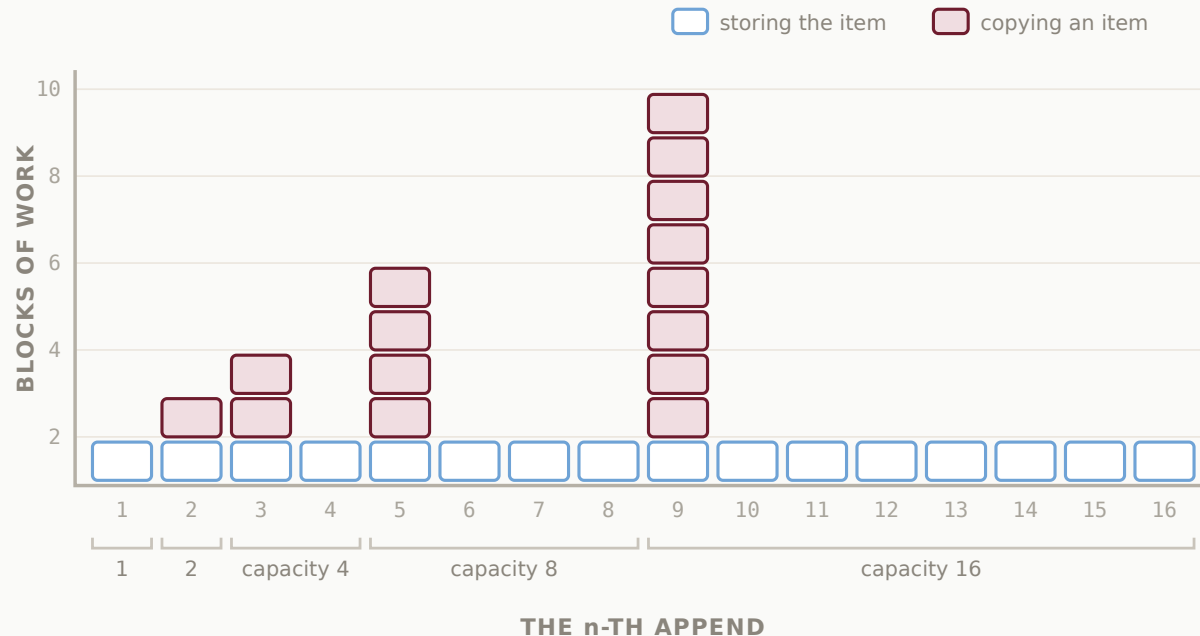
Amortized Constant Time

4 Append 9: the array is full, so it copies 8 items first.



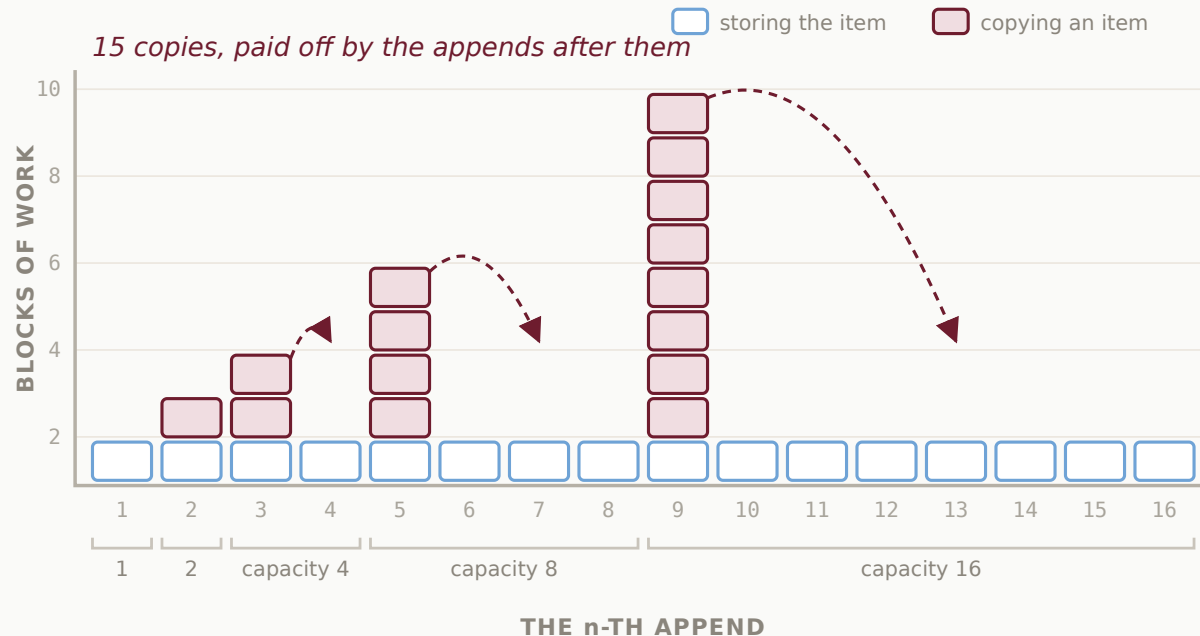
Amortized Constant Time

5 Append 16: there is room, so it just stores the item.



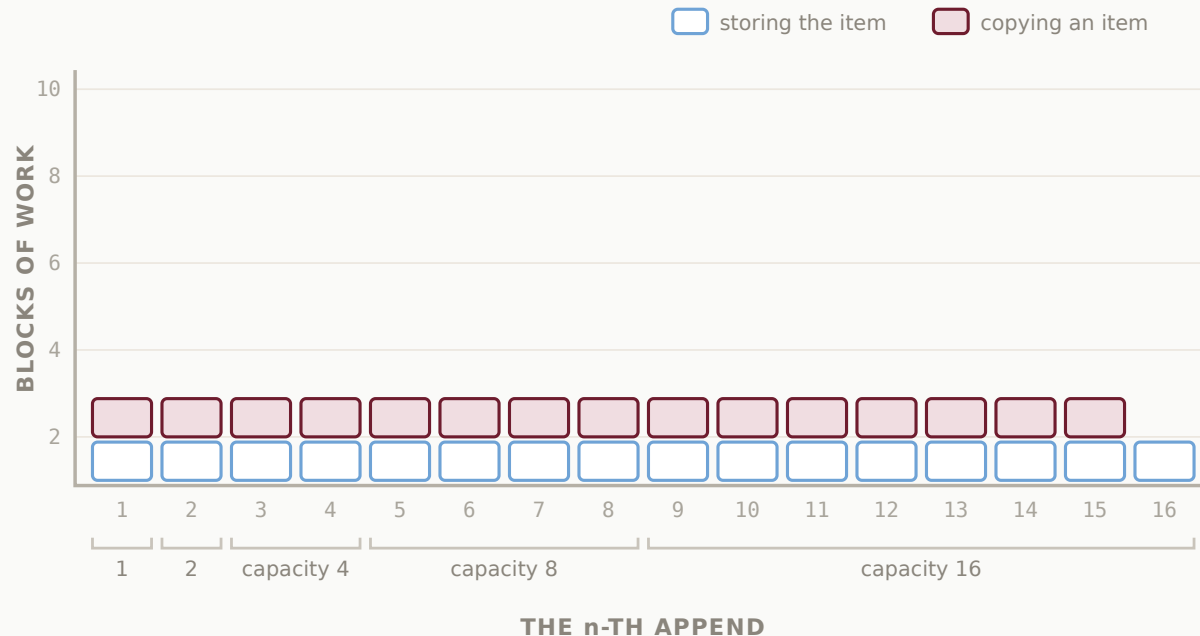
Amortized Constant Time

- 6 Each spike is paid for by the appends that follow it.



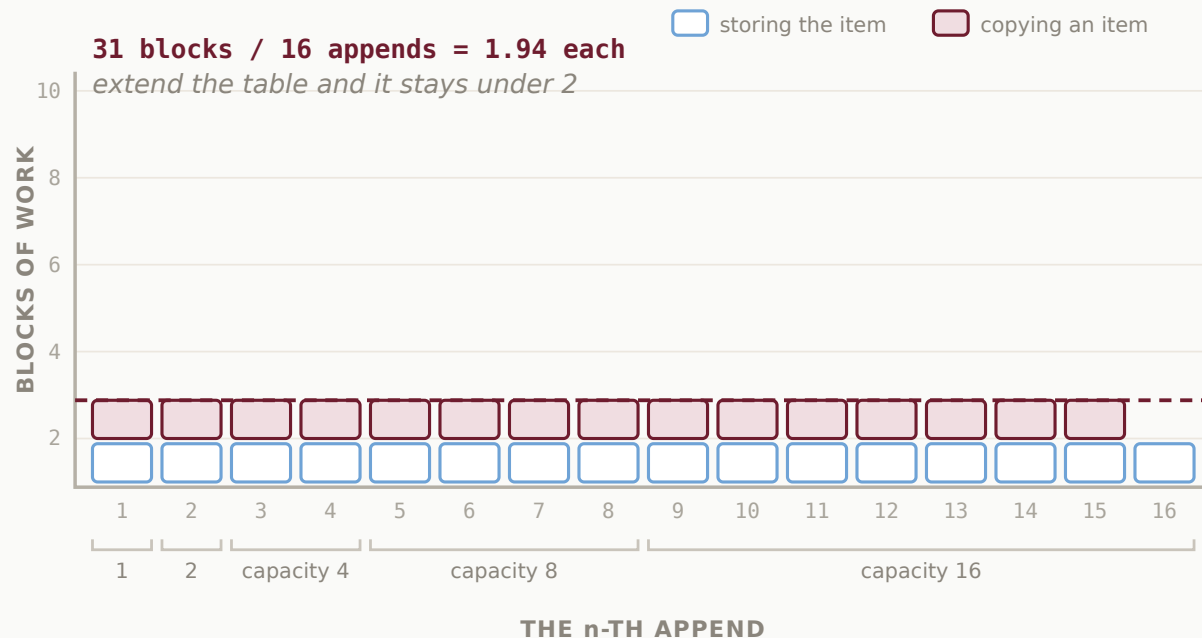
Amortized Constant Time

7 The same blocks, laid flat. Nothing was added or removed.



Amortized Constant Time

- 8 31 blocks over 16 appends, and the average does not grow with n .



The Same Thing in Algebra

Suppose the array is full, holding n items. Take the next n appends:

- The first one is the expensive one. Call its cost **n** .
- The other $n - 1$ find room. Call each one **1**.

total	$n + (n - 1) \times 1 = 2n - 1$
per append	$(2n - 1) / n = 2 - 1/n$

As n grows, $1/n$ shrinks and the average settles just under **2**. It does not grow with n , and that is what **amortized constant** means.

TALK TO YOUR NEIGHBOR · TTYN

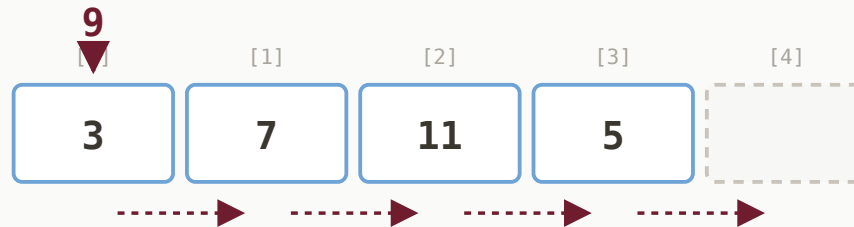
An earlier **append** grew the array by one slot each time it filled up. What did one append cost then?

- A.** Constant: it only adds one slot
- B.** Linear: every append copies the whole array
- C.** Amortized constant
- D.** It depends on the **Item** type

TALK TO YOUR NEIGHBOR · TTYN

Inserting at the *front* of a dynamic array is...

insert 9 at the front



*4 moves before the new value fits
and the last one moves first*

- A.** Constant time
- B.** Amortized constant time
- C.** Linear time, every element must shift up one
- D.** Faster than appending

Cost So Far

Operation	Dynamic array	Why
<code>v[i]</code>	constant	address arithmetic
<code>append()</code>	amortized constant	doubling spreads the copy
<code>insert()</code> at front	linear	everything shifts
<code>remove()</code> from middle	linear	everything after shifts
copy constructor	linear	copies every element
<code>getSize()</code>	constant	we stored the count

Keep this table. Next week we build a structure with almost the opposite profile, and the comparison is the point.

Week 05 Recap

- A **class template** takes a type the way a function takes an argument
- `PyList<int>` and `PyList<string>` are two separately generated classes
- Template implementations live in the **header**
- Every STL container is walked with `begin()` and `end()`
- Indexing is constant; append is **amortized constant**; inserting at the front is linear

Next: a06 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**

